Entregable 2.

- **Asignatura:** Fundamentos de Arquitectura de Software
- **Caso de Estudio:** Sistema Digital de Juego de Cartas (Variante Clásica & Variante Apuntado a 101)
- **Marco Teórico Base:** *Fundamentals of Software Architecture: A Modern Engineering Approach* (2.ª edición, 2025) – Mark Richards & Neal Ford.

Integrantes del Equipo:

Nombre Completo	**Doc. Identificación**	**Correo Electrónico**
Angelika Moya Bolaños	1061702724	amoyab@unal.edu.co
Daniel Restrepo Faciolince	1033266333	darestrepof@unal.edu.co
Thomas Monnier Granda	1038866661	tmonnier@unal.edu.co

Repositorio de Github: https://github.com/Thom204/ApuntadoFdA

Entregable 2.....1
1. Selección y Justificación del Estilo Arquitectónico.....3
1.1 Evaluación Comparativa de Estilos Candidatos.....3
1.2 Descripción del Estilo Seleccionado y Topología.....6
1.3 Diagrama UML de Componentes Detallado (Interfaces y Puertos).....7
2.1 Justificación de la Topología de Base de Datos.....8
2.2.2 Descripción de las tablas.....10
2.2.3 Estado persistido de la partida en curso.....14
2.3 Diagrama UML de Clases de Entidad y Mapeo de Persistencia.....15
2.4 Estrategia de Transaccionalidad y Consistencia de Puntuación.....16
3. Registro Formal de Decisiones Arquitectónicas (ADRs).....16
3.1 ADR-03: Monolito modular con módulo de reglas extensible por plug-ins en tiempo de ejecución.....16
3.2 ADR-04: Selección del Motor y Modelo de Persistencia (Relacional).....17
3.3 ADR-05: Comandos síncronos por interfaz y notificaciones por eventos post-commit.....18
3.4 Decisiones de Arquitectura Tecnológica del MVP.....19
3.4.1 Resumen de decisiones.....19
4. Análisis de Riesgo Arquitectónico y Risk Storming.....20
4.1 Matriz de Riesgo 3x3 (Impacto vs. Probabilidad).....20
4.2 Reporte de la Sesión de Risk Storming.....21
4.3 Plan de Mitigación de Riesgos Críticos (Zonas 6 a 9).....21
4.4 Riesgos Aceptados y Exclusiones del MVP.....22
5. Gobernanza y Funciones de Idoneidad (Architectural Fitness Functions).....22
5.1 Reglas de Gobernanza Estructural y Modularidad.....22
5.2 Implementación y Código de Fitness Functions.....23
5.2.1 Por qué GUT.....23
5.2.2 Estructura en el repositorio.....24
5.2.3 Cómo funciona el análisis.....24
5.2.4 Ejemplo: FF-01, aislamiento de capas.....24
5.2.5 Ejecución y alcance.....25
5.3 Límites de Complejidad Ciclomática y Reglas de Dependencias.....25
5.3.1 Complejidad ciclomática (FF-05).....25
5.3.2 Reglas de dependencias (FF-01 a FF-04).....26
5.4 Estrategia Mínima de Pruebas y Comprobaciones Arquitectónicas.....26

1. Selección y Justificación del Estilo Arquitectónico

1.1 Evaluación Comparativa de Estilos Candidatos

Para dar soporte a los atributos de calidad críticos identificados en el Entregable 1 (Configurabilidad/Extensibilidad DA-01, Testabilidad DA-02 y Responsiveness DA-03), se evaluaron dos estilos arquitectónicos candidatos de acuerdo con los lineamientos de Richards & Ford (Cap. 19): el estilo Microkernel (Plug-in) y el Monolito Modular.

> **Nota conceptual:** Un *plug-in* (también conocido como complemento o extensión) es un componente de software que se instala en una aplicación principal para añadirle nuevas funciones sin necesidad de modificar el código original del programa.

> **Nota metodológica sobre la tabla:** Las primeras tres filas de la evaluación utilizan una escala de 1 a 5 estrellas porque miden el grado de idoneidad frente a los atributos de calidad (drivers arquitectónicos). Las filas finales corresponden a características de costo operativo, propiedades topológicas y estructurales (particionamiento y quanta), las cuales se describen de forma conceptual y directa según la taxonomía del marco teórico al no tratarse de métricas de desempeño ordinales.

> **Nota sobre los candidatos:** la columna "Microkernel puro" evalúa la variante en la que turnos, mazo y puntaje también serían plug-ins y el núcleo quedaría casi vacío (alternativa C de ADR-03). El estilo seleccionado combina ambos candidatos y se describe en la síntesis que sigue a la tabla y en la sección 1.2.

Leyenda de la Escala de Calificación (1 a 5 estrellas):

- ⭐ (**1 estrella - Deficiente**): El estilo arquitectónico no soporta o dificulta severamente la característica.
- ⭐⭐ (**2 estrellas - Insuficiente**): El soporte es bajo y requiere adaptaciones complejas o costosas.
- ⭐⭐⭐ (**3 estrellas - Moderado / Bueno**): El estilo soporta la característica de forma aceptable, aunque presenta algunos compromisos (*trade-offs*).
- ⭐⭐⭐⭐ (**4 estrellas - Alto / Muy Bueno**): El estilo ofrece un desempeño sólido y natural para la característica.
- ⭐⭐⭐⭐⭐ (**5 estrellas - Excelente / Óptimo**): El estilo está diseñado específicamente para maximizar y blindar esta característica de calidad.

Característica de Calidad / Criterio	Microkernel Puro (Plug-in)	Monolito Modular	Justificación de la Calificación y Trade-offs
Extensibilidad de Reglas (DA-01)	⭐⭐⭐⭐⭐ (5/5)	⭐⭐⭐⭐ (4/5)	El **Microkernel puro** aísla cada variante en un plug-in independiente y el núcleo no conoce ninguna regla concreta. El **Monolito Modular** alcanza un nivel alto con Strategy + Registry: no modifica el orquestador, pero cada variante nueva

Característica de Calidad / Criterio	Microkernel Puro (Plug-in)	Monolito Modular	Justificación de la Calificación y Trade-offs
			modifica el registro y el catálogo de modalidades y obliga a re-exportar el servidor (ADR-02).
Testabilidad Aislada (DA-02)	⭐⭐⭐⭐ (4/5)	⭐⭐⭐⭐⭐ (5/5)	En el **Monolito Modular**, la lógica de dominio pura (GameRule, HandEvaluator) se prueba *headless* en milisegundos mediante inyección de dependencias, sin UI ni BD. En el **Microkernel puro**, como turnos, mazo y puntaje también serían plug-ins, cada flujo de prueba debe montar el cargador y validar contratos entre varios plug-ins, lo que agrega preparación y puntos de falla.
Responsiveness y Latencia (DA-03)	⭐⭐⭐⭐ (4/5)	⭐⭐⭐⭐⭐ (5/5)	Ambos estilos se ejecutan en un solo proceso y en memoria. El **Monolito Modular** resuelve cada comando con llamadas directas entre módulos. En el **Microkernel puro**, cada paso del turno (robar, descartar, cerrar) cruzaría el contrato de un plug-in, con más indirección y validaciones en el camino crítico de ≤ 200 ms. La diferencia es pequeña, por eso recibe 4 y no menos.
Simplicidad y Costo Operativo	⭐⭐⭐ (3/5)	⭐⭐⭐⭐⭐ (5/5)	El **Monolito Modular** no gestiona contratos, versiones ni empaquetado de plug-ins. En el **Microkernel puro**, todo el flujo del juego pasa por contratos versionados y cada parte se empaqueta por separado. Richards y Ford destacan la simplicidad del microkernel cuando el núcleo concentra la funcionalidad; esa ventaja se pierde cuando el núcleo queda casi vacío.
Particionamiento General	Por Dominio y Técnico	Por Dominio	El **Microkernel puro** combina un núcleo particionado técnicamente con plug-ins por dominio. El **Monolito Modular** se particiona por dominio puro, alineado con los límites del negocio del juego (res://rules, res://turn, res://match).
Quanta Arquitectónico	1 Quantum	1 Quantum	Al tratarse de una aplicación centrada en salas de 5 jugadores dentro de una unidad lógica unificada, un solo quantum

Característica de Calidad / Criterio	Microkernel Puro (Plug-in)	Monolito Modular	Justificación de la Calificación y Trade-offs
			arquitectónico es óptimo y suficiente para el rendimiento en tiempo real.

El Microkernel puro obtiene la mejor calificación en extensibilidad porque aísla cada variante en un plug-in. El Monolito Modular lo supera en testabilidad, responsiveness y simplicidad, porque toda interacción es una llamada en proceso y no exige probar un cargador en cada flujo. Se adopta un híbrido (ADR-03). El núcleo (orquestación, turnos, mazo y puntaje) es estable y lo comparte toda la familia de juegos de combinaciones, así que se mantiene como monolito modular. Solo el módulo de reglas, el único punto volátil, se organiza como microkernel interno. Así se conserva la extensibilidad del microkernel (2 archivos nuevos, 0 modificados, sin recompilar el núcleo) sin penalizar la responsiveness: los `.pck` se cargan una sola vez al arrancar y cada validación es una llamada a método en memoria (≤ 200 ms, DA-03). Se aceptan como costo un contrato versionado difícil de cambiar, reiniciar el servidor para agregar un plug-in y plug-ins sin *sandbox*.

1.2 Descripción del Estilo Seleccionado y Topología

Se selecciona una arquitectura de **Monolito Modular** estructurada por dominio (ADR-01), con un **microkernel interno en el módulo de reglas** (ADR-03), desplegada como una única unidad ejecutable (1 *quantum*). El sistema se descompone en módulos lógicos desacoplados, cada uno delimitado formalmente dentro del árbol del proyecto mediante *namespaces* aislados bajo el directorio raíz *`res://`*:

- ***`res://match/ (apuntado.match):`*** Administra el ciclo de vida de la partida, configuración (setup) y determinación de estado final (outcome).
- ***`res://turn/ (apuntado.turn):`*** Modela el orquestador circular de turnos, fases (DrawPhase, DiscardPhase) y el estado de la ronda mediante el patrón *State*.
- ***`res://rules/ (apuntado.rules):`*** Motor de reglas organizado como microkernel interno. Contiene el contrato GameRule (res://rules/game_rule.gd), la evaluación algorítmica de combinaciones (evaluation), el registro de plug-ins (selection) y el punto de montaje de los plug-ins de cada modalidad (modes/<id>/).
- ***`res://deck/ (apuntado.deck):`*** Encapsula la reserva de cartas, barajado e invariantes de conservación de los mazos y pozos de descarte.
- ***`res://scoring/ (apuntado.scoring):`*** Gestiona la liquidación matemática de penalizaciones y acumulados globales de puntuación.
- ***`res://notification/ (apuntado.notification):`*** Canales de difusión de eventos hacia la interfaz mediante el patrón *Observer*.
- ***`res://history/ (apuntado.history):`*** Registro cronológico e inmutable de movimientos de la partida.

Mecanismo de Coordinación (Mediator): Para evitar dependencias directas o cruzadas entre estos módulos, la coordinación del ciclo de vida se resuelve mediante el componente MatchOrchestrator (`apuntado.orchestration`), ubicado en *`res://orchestration/`*. Este componente actúa como un Mediator (Cap. 8, Richards & Ford): recibe las solicitudes e intenciones de la capa de entrada/red, invoca secuencialmente a las interfaces públicas de los módulos colaboradores (como Turn Orchestration o Round Scoring) y retransmite los

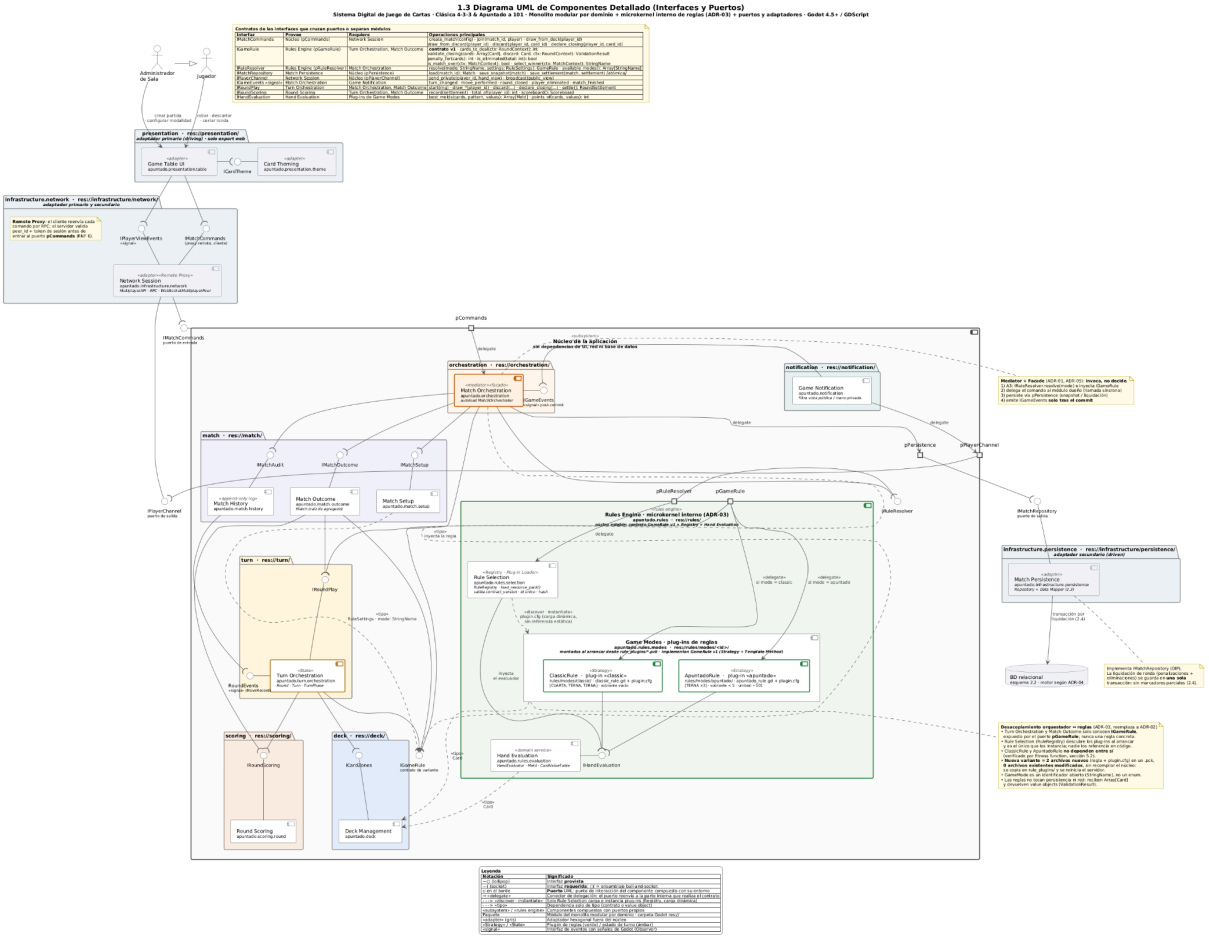
eventos resultantes. Ningún módulo conoce o llama directamente a otro de manera horizontal; únicamente responden al orquestador central o emiten eventos asíncronos.

Motor de reglas como microkernel interno (ADR-03). El módulo rules funciona como un microkernel dentro del monolito (Richards y Ford, cap. 13). El core system es el resto del monolito más el contrato común GameRule v1 (res://rules/game_rule.gd), versionado. Los plug-ins ClassicRule y ApuntadoRule son paquetes .pck de 2 archivos (regla y plugin.cfg) montados en res://rules/modes/<id>/. El registro RuleRegistry los carga al arrancar, válida hash y versión del contrato y resuelve la regla según mode. Una modalidad nueva requiere 2 archivos nuevos, 0 modificados y ninguna recompilación del núcleo (OCP, RNF 1).

Limitaciones y Gobernanza: Al compartir el mismo espacio de memoria y proceso de ejecución, el aislamiento modular no lo impone el compilador, sino la disciplina arquitectónica. Este aislamiento se garantiza activamente mediante reglas de dependencia basadas en class_name, prohibición de acceso por rutas de nodos absolutos entre carpetas y funciones *fitness* automáticas ejecutadas en integración continua (CI) que detectan ciclos entre *namespaces*.

1.3 Diagrama UML de Componentes Detallado (Interfaces y Puertos)

Se detalla la estructuración topológica especificando los puntos de acoplamiento formales de entrada y salida entre la lógica central y las entidades calificadoras de partidas:



Fragmento 1.3. Diagrama UML de Componentes Detallado (Interfaces y Puertos). Link: https://github.com/Thom204/ApuntadoFdA/blob/main/docs/diagrams/component-diagram-detailed.svg

2. Arquitectura de Información y Topología de Datos

Esta sección describe cómo se almacena y se protege el estado del juego. Se organiza en cuatro partes: la elección de la topología de base de datos (2.1), el esquema relacional con sus tablas, relaciones y restricciones (2.2), la correspondencia entre las clases del dominio y las entidades de persistencia (2.3) y la estrategia de transacciones que mantiene consistente la puntuación (2.4).

Un principio gobierna todo el diseño: el servidor es la única autoridad sobre el estado de cada partida (D-03). Los clientes nunca acceden a la base de datos; esta es el medio de persistencia y recuperación del servidor, no un recurso compartido.

2.1 Justificación de la Topología de Base de Datos

La topología de almacenamiento se eligió a partir de cuatro requisitos que impone el juego:

- **Persistencia fiable** de las partidas, de las manos privadas de los jugadores y del historial de rondas.
- **Liquidación atómica** de cada ronda: calcular las penalizaciones de los jugadores que no cerraron, actualizar el marcador y verificar el umbral de eliminación (-101 puntos) como una sola operación indivisible.
- **Recuperación** de las partidas en curso tras un reinicio del servidor (D-15).
- **Acceso exclusivo del servidor:** al existir una única autoridad (D-03 y D-04), no hay que sincronizar bases de datos entre clientes.

Con base en estos requisitos se compararon cuatro alternativas, valorando el equilibrio entre consistencia y simplicidad operativa:

Alternativa	Fortalezas	Limitaciones para este sistema	Decisión
Relacional con servidor de BD dedicado (PostgreSQL)	Garantías ACID, integridad referencial, esquema en Tercera Forma Normal (3FN) y relación 1:N natural entre partida y rondas.	Agrega un proceso y su operación (red, credenciales, respaldos) sin beneficio mientras exista un único proceso escritor (D-03, ADR-05).	Descartada
Embebida en cliente (SQLite)	Costo de infraestructura nulo y despliegue simple, sin servidor de base de datos.	La fuente de verdad reside en un servidor central. Una base embebida en cada cliente obligaría a resolver conflictos entre estados divergentes sin aportar ningún beneficio.	Descartada
Mixto (Embebida en server-side)	Una base de datos embebida en server-side, nos permite tener el grado de soporte nativo e integración de una base de datos embebida sin sacrificar la	Un solo escritor; escalar a varias instancias del servidor exigiría migrar a un servidor de BD, lo que el puerto IMatchRepository permite sin tocar el núcleo.	**Aceptada**

	separación de roles puesto que esta pertenece al proceso server		
Documental NoSQL (p. ej., MongoDB)	Flexibilidad para almacenar estructuras anidadas, como las manos de cartas o los historiales dentro del documento de la partida.	Sin consistencia transaccional nativa entre colecciones: la atomicidad del cálculo de puntajes y eliminaciones recaería en el código de la aplicación, con mayor complejidad.	Descartada

Veredicto. Se adopta una topología monolítica centralizada relacional, implementada con SQLite embebida en el proceso servidor (ADR-04). Es centralizada porque la base existe solo en el servidor y solo el servidor escribe en ella; los clientes nunca la acceden. Prioriza la integridad atómica, la consistencia y la exactitud del marcador sobre la flexibilidad del esquema. Se aceptan un DDL rígido, migraciones explícitas y un único escritor; el puerto IMatchRepository aísla el motor, de modo que migrar a un servidor de BD no modifica el núcleo.

2.2 Especificación del Esquema de Datos

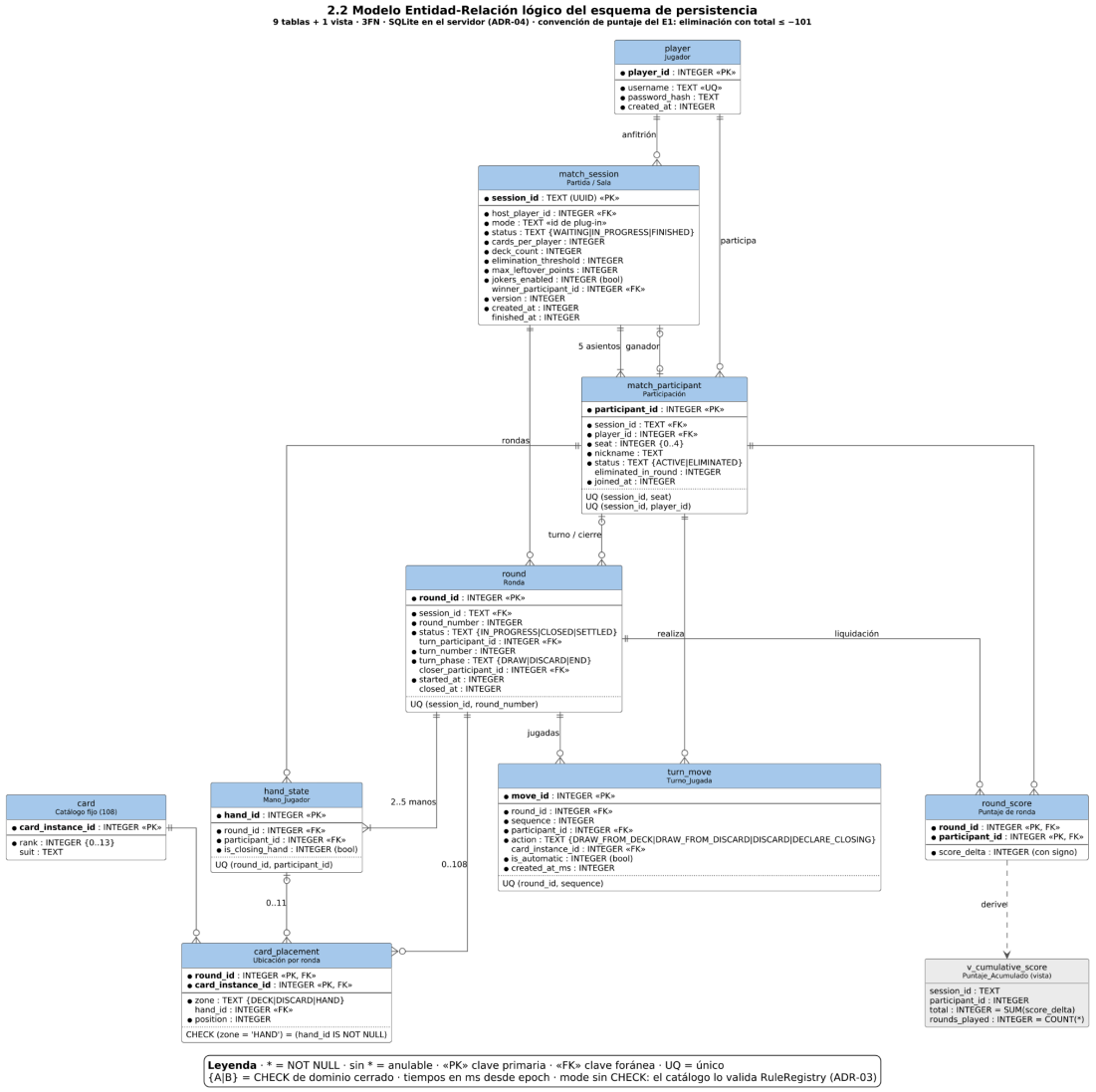


Figura 2.2. *Modelo entidad-relación lógico del esquema de persistencia.* Diagrama completo: https://github.com/Thom204/ApuntadoFdA/blob/main/docs/diagrams/er-diagram.svg

El esquema desarrolla la **Opción A** (relacional, modelo E-R); la Opción B (no relacional) se descartó en la sección 2.1. Está normalizado en Tercera Forma Normal (3FN) y consta de **nueve tablas y una vista**, cada una con su entidad de persistencia en el diagrama 2.3. Cubre las seis tablas mínimas de la guía: Jugador (player), Partida (match_session), Ronda (round), Mano_Jugador (hand_state), Turno_Jugada (turn_move) y Puntaje_Acumulado (v_cumulative_score). Los tipos se indican a nivel lógico; en SQLite (ADR-04), los enteros y booleanos se guardan como INTEGER y los textos como TEXT.

Relación	Cardinalidad	Significado
player — match_participant	1 : 0..N	Un jugador participa en varias partidas; cada participación es una fila.
player — match_session (host_player_id)	1 : 0..N	Un jugador puede ser anfitrión de varias salas.
match_session — match_participant	1 : 5	Una sala admite exactamente 5 participantes (D-03).
match_session — round	1 : 0..N	Clásica: una ronda. Apuntado: rondas sucesivas hasta que quede un jugador o se cumpla RD08. Cero mientras la sala espera jugadores.
match_participant — match_session (winner_participant_id)	0..1 : 0..1	Ganador de la partida; vacío mientras no termina.
match_participant — round (turn_participant_id, closer_participant_id)	1 : 0..N	La ronda registra a quién le toca y quién la cerró.
round — hand_state	1 : 2..5	Una mano por participante activo en la ronda.
match_participant — hand_state	1 : 0..N	Cada mano pertenece a un participante.
round — card_placement	1 : 0..108	Ubicación de cada carta de los dos mazos en la ronda.
card — card_placement	1 : 0..N	Cada carta del catálogo se ubica una vez por ronda.
hand_state — card_placement	0..1 : 0..11	Cartas que están en esa mano.
round — turn_move	1 : 0..N	Jugadas de la ronda, en orden.
match_participant — turn_move	1 : 0..N	Quién hizo cada jugada.

round — round_score	1 : 0..5	Al liquidar la ronda se guarda una fila de puntaje por participante.
match_participant — round_score	1 : 0..N	Puntajes del participante en cada ronda.
round_score ⟶ v_cumulative_score	derivación	La vista suma los score_delta de cada participante.

2.2.2 Descripción de las tablas

player · Jugador

Atributo	Tipo lógico	Restricción	Descripción
player_id	entero	PK	Identifica al jugador.
username	texto	UNIQUE, NOT NULL	Nombre de usuario.
password_hash	texto	NOT NULL	Credencial almacenada como hash (D-08).
created_at	entero (ms)	NOT NULL	Fecha de registro.

`match_session` · Partida / Sala

Atributo	Tipo lógico	Restricción	Descripción
session_id	texto (UUID)	PK	Identifica la partida y su sala.
host_player_id	entero	FK ⟶ player, NOT NULL	Anfitrión que creó la sala.
mode	texto	NOT NULL	Identificador del plug-in de reglas activo (“classic”, “apuntado”). No lleva CHECK ni enumeración: un CHECK obligaría a migrar el esquema con cada variante nueva (ADR-03). RuleRegistry valida el id al crear la sala.
status	texto	CHECK (WAITING, IN_PROGRESS, FINISHED)	Ciclo de vida de la sala y la partida (D-16).
cards_per_player	entero	NOT NULL	Cartas repartidas por jugador.
deck_count	entero	NOT NULL	Número de mazos (2).
elimination_threshold	entero	NOT NULL	Umbral de eliminación, copiado de los valores por defecto del plug-in al crear la sala (Apuntado: −101). La modalidad Clásica no lo usa.

max_leftover_points	entero	NOT NULL	Máximo de puntos sobrantes para cerrar (Apuntado: menos de 5).
jokers_enabled	booleano	NOT NULL	Habilita los comodines.
winner_participant_id	entero	FK → match_participant, anulable	Ganador; vacío mientras la partida no termina.
version	entero	NOT NULL	Versión de la partida. Aumenta con cada cambio aceptado y sostiene el control optimista y la idempotencia (D-07, sección 2.4).
created_at	entero (ms)	NOT NULL	Fecha de creación de la sala.
finished_at	entero (ms)	anulable	Fin de la partida.

`match_participant` · Participación (tabla asociativa N:M entre `player` y `match_session`)

Atributo	Tipo lógico	Restricción	Descripción
participant_id	entero	PK	Identifica la participación de un jugador en una partida.
session_id	texto (UUID)	FK → match_session, NOT NULL	Partida en la que participa.
player_id	entero	FK → player, NOT NULL	Jugador que participa.
seat	entero	CHECK (0–4)	Asiento; fija el orden circular de los turnos.
nickname	texto	NOT NULL	Alias mostrado en esa partida.
status	texto	CHECK (ACTIVE, ELIMINATED)	Estado del participante. ELIMINATED corresponde al rol de espectador (D-08).
eliminated_in_round	entero	anulable	Número de la ronda en que quedó eliminado, por puntaje o por inactividad (D-12).
joined_at	entero (ms)	NOT NULL	Momento en que ingresó a la sala.

Restricciones de tabla: UNIQUE (session_id, seat) y UNIQUE (session_id, player_id): un jugador no ocupa dos plazas ni dos asientos en la misma sala. La capacidad de exactamente 5 participantes es una regla de negocio que el servidor valida al admitir jugadores (D-03).

`round` · Ronda

Atributo	Tipo lógico	Restricción	Descripción

round_id	entero	PK	Identifica la ronda.
session_id	texto (UUID)	FK → match_session, NOT NULL	Partida a la que pertenece.
round_number	entero	NOT NULL	Posición de la ronda dentro de la partida.
status	texto	CHECK (IN_PROGRESS, CLOSED, SETTLED)	En juego, cerrada por un jugador o liquidada.
turn_participant_id	entero	FK → match_participant, anulable	Participante al que le toca jugar.
turn_number	entero	NOT NULL	Contador de turnos de la ronda.
turn_phase	texto	CHECK (DRAW, DISCARD, END)	Fase del turno actual (patrón State).
closer_participant_id	entero	FK → match_participant, anulable	Participante que cerró la ronda; vacío mientras no hay cierre.
started_at	entero (ms)	NOT NULL	Inicio de la ronda.
closed_at	entero (ms)	anulable	Momento del cierre.

`hand_state` · Mano_Jugador

Atributo	Tipo lógico	Restricción	Descripción
hand_id	entero	PK	Identifica la mano.
round_id	entero	FK ⟶ round, NOT NULL	Ronda a la que corresponde.
participant_id	entero	FK ⟶ match_participant, NOT NULL	Propietario de la mano.
is_closing_hand	booleano	NOT NULL	Marca la mano con la que se cerró la ronda.

turn_move · Turno_Jugada

Atributo	Tipo lógico	Restricción	Descripción
card		card_instance_id (PK), rank (0–13; 0 = comodín), suit (anulable). 108 filas fijas.	Identifica cada carta de los dos m
card_placement		PK (round_id, card_instance_id), ambas FK; zone (DECK, DISCARD, HAND); hand_id (FK ⟶ hand_state, solo si zone = HAND); position.	Reemplaza las columnas JSON (1FN). La PK compuesta garantiza que cada carta esté en un solo lugar por ronda, y

			permite reconstruir mazo, descarte y manos al reconectar.
turn_move		move_id (PK); round_id (FK) y sequence, únicos juntos; participant_id (FK); action (DRAW_FROM_DECK, DRAW_FROM_DISCARD, DISCARD, DECLARE_CLOSING); card_instance_id (FK, anulable); is_automatic; created_at_ms.	Historial de acciones (D-14), solo inserciones. is_automatic permite contar los turnos automáticos (D-12).

`round_score` · Puntaje de ronda

Atributo	Tipo lógico	Restricción	Descripción
round_id	entero	PK, FK → round	Ronda liquidada.
participant_id	entero	PK, FK → match_participant	Participante al que corresponde el puntaje.
score_delta	entero con signo	NOT NULL	Variación del puntaje en la ronda: negativa para cada perdedor (valor de sus cartas no combinadas) y +10 para quien cierra (RF07).

La clave compuesta (round_id, participant_id) garantiza una sola fila por participante y ronda, e impide liquidar la misma ronda dos veces.

`v_cumulative_score` · Vista de puntaje acumulado

Atributo	Tipo lógico	Derivación	Descripción
session_id	texto (UUID)	match_participant.session_id	Partida.
participant_id	entero	round_score.participant_id	Participante.
total	entero	SUM(score_delta)	Puntaje acumulado en la partida.
rounds_played	entero	COUNT(*)	Rondas liquidadas del participante.

Por qué el acumulado es una vista. El puntaje acumulado es un dato derivado. Guardarlo como columna duplicaría lo que ya contienen los score_delta, podría quedar inconsistente con ellos y crearía una dependencia transitiva que viola la 3FN. Por eso se calcula con la vista, que la liquidación consulta para verificar el umbral de eliminación (sección 2.4).

2.2.3 Estado persistido de la partida en curso

Las decisiones D-13 y D-14 (sección 3.4.5) exigen que la base de datos conserve el estado necesario para recuperar una partida en curso y un historial de acciones. La tabla indica dónde se almacena cada elemento en el esquema descrito:

Elemento que se debe persistir	Ubicación en el esquema
Estado del juego (ciclo de vida, D-16)	match_session.status, finished_at
Configuración de la partida (modalidad, umbral y parámetros de la regla)	match_session.mode, elimination_threshold, max_leftover_points, jokers_enabled, cards_per_player, deck_count
Versión de la partida (D-07)	match_session.version
Estado de los participantes y rol de espectador (D-08)	match_participant.status (ELIMINATED = espectador), eliminated_in_round
Manos de los jugadores	hand_state + card_placement (zone = HAND)
Mazo y pila de descarte	card_placement (zone = DECK o DISCARD, en el orden de position)
Rondas y su cierre	round.status, closer_participant_id, closed_at
Turno actual	round.turn_participant_id, turn_number, turn_phase
Puntuaciones	round_score.score_delta y vista v_cumulative_score
Historial de acciones y eventos (D-14)	turn_move (solo inserciones, ordenado por sequence)
Turnos automáticos consecutivos (D-12)	Se derivan de turn_move.is_automatic: se cuentan las jugadas automáticas consecutivas del participante. Los parámetros del temporizador son configuración del servidor (D-19), no datos de la partida.

2.3 Diagrama UML de Clases de Entidad y Mapeo de Persistencia

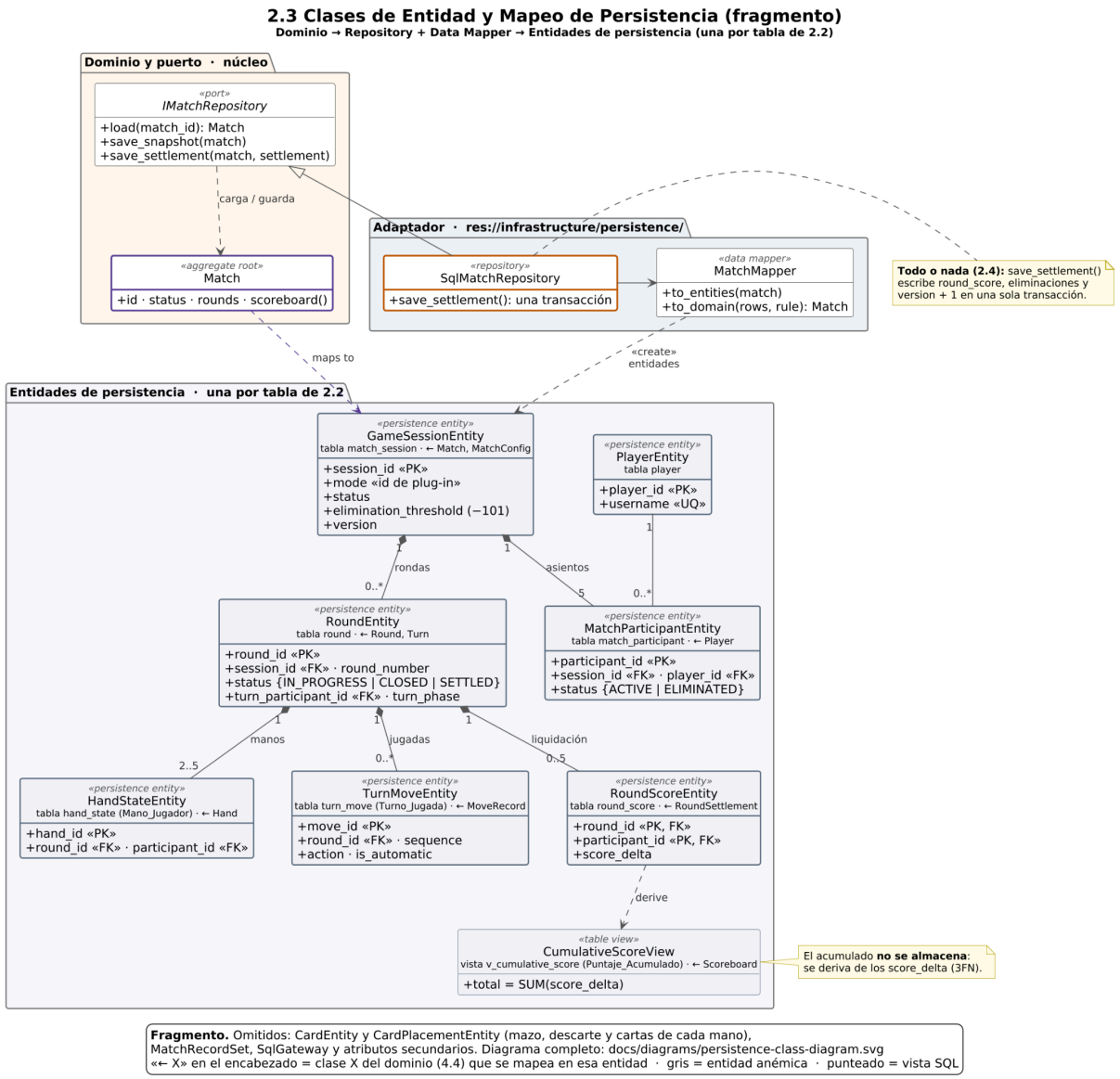


Figura 2.3. Fragmento del diagrama de clases de persistencia. Diagrama completo: https://github.com/Thom204/ApuntadoFdA/blob/main/docs/diagrams/persistence-class-diagram.svg

2.4 Estrategia de Transaccionalidad y Consistencia de Puntuación

La liquidación de una ronda se protege con tres barreras.

1. **Serialización (ADR-05):** el servidor ejecuta en serie los comandos de cada partida, así que dos cierres nunca se procesan a la vez; el segundo encuentra la ronda en CLOSED y se rechaza.
2. **Una transacción ACID:** save_settlement() abre BEGIN IMMEDIATE en SQLite. Incrementa match_session.version solo si coincide con la versión leída, pasa la ronda a SETTLED solo si no lo estaba, inserta una fila de round_score por participante (la PK compuesta impide duplicados) y marca como ELIMINATED a quien quede con

total ≤ −101 en v_cumulative_score. Ante cualquier fallo hace ROLLBACK completo, así que nunca quedan marcadores parciales.

3. **Eventos posteriores al commit (ADR-05):** las notificaciones se emiten sólo después del COMMIT; si la transacción falla, no se notifica nada y el estado se recarga desde la base de datos.

3. Registro Formal de Decisiones Arquitectónicas (ADRs)

3.1 ADR-03: Monolito modular con módulo de reglas extensible por plug-ins en tiempo de ejecución

ESTADO: Accepted, reemplaza a ADR-02.

CONTEXTO: Agregar una variante (por ejemplo, Continental) debe crear o modificar como máximo 2 archivos, sin recompilar ni alterar el orquestador (DA-01), sin superar 200 ms (DA-03) y sin salir del monolito (DA-04). La Strategy en compilación de ADR-02 obliga a modificar el registro y re-exportar el servidor, y su contrato no conoce la ronda ni el marcador.

DECISIÓN: Se mantiene el monolito modular (ADR-01) y el módulo rules se convierte en un **microkernel interno** (Richards y Ford, cap. 13, fig. 13-2):

- **Contrato GameRule v1, versionado,** en el núcleo (`res://rules/game_rule.gd`). Recibe `RoundContext` y `MatchContext` y aplica Strategy + Template Method.
- **Plug-ins:** cada modalidad es un .pck con 2 archivos, la regla y `plugin.cfg`, que se monta al arrancar en `res://rules/modes/<id>/`. Clásica y Apuntado tienen el mismo formato.
- **Registry:** `RuleRegistry` verifica el hash antes de montar cada paquete, lo carga con `load_resource_pack(path, false)`, valida la versión del contrato y el id, y resuelve la regla según mode (`StringName`, catálogo abierto).
- **Alcance de v1:** los parámetros se limitan a `RuleSettings`. Póker, las apuestas dinámicas y el reingreso de eliminados quedan fuera, porque cambian el turno o el ciclo de vida del participante (*volatile core*).

Agregar Continental: escribir `continental_rule.gd` y `plugin.cfg`, pasar las pruebas de contrato, copiar el `.pck` en `rule_plugins/` y reiniciar el servidor.

CONSECUENCIAS:

- *Positivas:* 2 archivos nuevos y 0 modificados, sin recompilar ni redesplegar el núcleo. El orquestador, la persistencia y la UI no conocen las variantes. Cada plug-in se prueba de forma aislada.
- *Negativas:* el contrato es costoso de cambiar (por eso está versionado). Los plug-ins no tienen *sandbox*: solo el administrador los instala, con hashes aprobados. Agregar uno exige reiniciar el servidor y no se puede retirar con partidas activas.

CUMPLIMIENTO (COMPLIANCE):

- *Automatizado (GUT en CI, sección 5.2):* pruebas de contrato por plug-in con p95 ≤ 150 ms; el núcleo no referencia de forma estática `res://rules/modes/` (FF-01); cero dependencias entre plug-ins (FF-02); *volatility check* en los PR de plug-ins.
- *En ejecución:* `RuleRegistry` rechaza y audita los paquetes con hash no aprobado, versión de contrato distinta o id duplicado.

NOTAS: Daniel Restrepo, 24-10-2026. Versión completa, con las alternativas evaluadas: `docs/adrs/0003-reglas-como-plugins-en-tiempo-de-ejecucion.md`.

3.2 ADR-04: Selección del Motor y Modelo de Persistencia (Relacional)

ESTADO: Accepted.

CONTEXTO: Se requiere trazabilidad transaccional estricta para guardar el historial de partidas, manos privadas y el conteo acumulativo multironda de las penalizaciones hasta -101 puntos.

DECISIÓN: Seleccionar una base de datos **Relacional (SQL)** con un diseño normalizado en Tercera Forma Normal (3FN), el motor escogido es SQLite en server side, todas las operaciones en la DB son realizadas por el proceso servidor el cual es responsable de aplicar las validaciones y políticas correspondientes para transacciones duplicadas, inválidas u obsoletas. Se plantea un sistema centralizado y alojado en un servicio VPS a definir, esto nos permite que la instancia del servidor utilice una base de datos embebida como SQLite y que siga respetando la arquitectura de BD centralizada, esto debido a que pese a ser una base embebida, el único proceso que tiene permiso para escribir en la misma es el servidor, por lo cual el archivo con los datos sólo vive en la VPS que aloja al servidor y las instancias de proceso cliente, que existen en el navegador de los jugadores solo realizan transacciones comunicándose con el proceso server.

CONSECUENCIAS:

- *Positivas:* Asegura la integridad ACID en el cálculo simultáneo de puntajes de los 5 jugadores, evitando divergencias financieras o aritméticas en el marcador. El hecho de tener un único proceso escritor permite evitar condiciones de carrera a nivel de partida y garantiza tener una única fuente de verdad para el estado de la partida sin necesidad de validarlo constantemente contra la BD.

- *Negativas:* requiere esquemas migratorios estrictos ante cambios en la estructura de datos y plantea desafíos adicionales de escalabilidad en el escenario donde sea necesario tener múltiples instancias del servidor, sea por balance de carga o por redundancia, estos últimos aunque quedan fuera del scope de esta entrega, puede mitigarse encapsulando correctamente el módulo de transacciones de modo que se puedan implementar migraciones y mecanismos de escalada cuando sea necesario.

COMPLIANCE: Verificado mediante revisiones de esquemas DDL en el repositorio que validan el uso de llaves foráneas y restricciones de unicidad por sala, adicionalmente el código del módulo que gestiona las transacciones de BD será auditado a fondo en la fase de testeo.

3.3 ADR-05: Comandos síncronos por interfaz y notificaciones por eventos post-commit

ESTADO: Accepted.

CONTEXTO: Los comandos (robar, descartar, cerrar) necesitan una respuesta inmediata solo para el solicitante, en ≤ 200 ms (DA-03). Las notificaciones (descarte, cambio de turno, cierre, eliminación) deben llegar a los 5 clientes, cada uno con su vista (RNF 3), sin que el emisor conozca a los receptores. En Godot, el servidor procesa las RPC en el hilo principal, y la asincronía de la red no debe filtrarse al núcleo, para mantener el determinismo y las pruebas *headless* (DA-02).

DECISIÓN: Esquema híbrido: síncrono por defecto, asíncrono solo cuando hace falta (Richards y Ford, cap. 19).

1. Comandos síncronos en memoria a través de interfaces, coordinados por Match Orchestration como mediador. El orden es fijo: validar sesión → Turn.can() → ejecutar en el dominio → persistir → emitir eventos. Las llamadas a los plug-ins de reglas también son síncronas y en proceso (ADR-03).
2. Notificaciones por eventos post-commit (Observer con señales). Solo el orquestador emite IGameEvents, y solo después de guardar; si la persistencia falla, no se emite nada.
3. Un solo hilo por servidor: los comandos de cada partida se ejecutan en serie, sin bloqueos; la columna version es la segunda barrera (2.4).
4. La red queda en el adaptador: sólo Network Session usa @rpc o multiplayer; el núcleo no usa await ni hilos.

CONSECUENCIAS:

- *Positivas:* respuesta inmediata y dirigida, orden determinista sin cierres simultáneos, consumidores nuevos sin modificar el orquestador y un núcleo que se prueba sin red.
- *Negativas:* todas las salas comparten el hilo principal, así que una escritura lenta o un plug-in lento retrasan a las demás (mitigación: transacciones pequeñas y p95 medido en CI). El orquestador es el punto de mayor acoplamiento (Ce = 6), un costo

aceptado del enfoque *mediator*.

CUMPLIMIENTO (COMPLIANCE): *Automatizado (GUT headless, sección 5.2):* el núcleo no contiene @rpc, multiplayer, await ni Thread; solo res://orchestration/ emite IGameEvents; si save_snapshot() falla, no se emite turn_changed; declare_closing() completo con p95 ≤ 200 ms.

NOTAS: Daniel Restrepo, 27-10-2026. Relacionado con ADR-01, ADR-03 y ADR-04 y con las secciones 1.3 y 2.4. Versión completa: `docs/adrs/0005-comandos-sincronos-y-eventos-post-commit.md`.

3.4 Decisiones de Arquitectura Tecnológica del MVP

Esta sección consolida las decisiones de arquitectura e implementación técnica acordadas para el MVP del Sistema Digital de Juego de Cartas (variantes Clásica y Apuntado a 101). El objetivo es mantener una arquitectura modular, con autoridad centralizada en el servidor, recuperación práctica ante desconexiones y mecanismos de validación y prueba proporcionados al alcance demostrativo de la entrega.

Las decisiones complementan a ADR-03, ADR-04 y ADR-05 y a las secciones 1 y 2, sin sustituirlas. En particular, no se reabren la selección del motor de base de datos ni el modelo de datos (ADR-04 y sección 2), y tampoco se fijan las reglas del dominio, que corresponden al responsable de la especificación del juego (ver 3.4.7). ADR-05 regula la comunicación entre los componentes internos del monolito; la comunicación entre cliente y servidor se define aquí (D-06).

Cada decisión se identifica con un código **D-01** a **D-23** y se agrupa por tema. Las decisiones sobre pruebas y comprobaciones automáticas (D-22 y D-23) se desarrollan en la sección 5.4, y las exclusiones explícitas del MVP en la sección 4.4.

3.4.1 Resumen de decisiones

ID	Decisión	Tema	Sección
D-01	Monolito modular, sin microservicios	Estructura	1.2
D-02	Variabilidad de reglas mediante plug-ins (Strategy sobre el contrato GameRule)	Estructura	1.2 · ADR-03
D-03	El servidor es la única autoridad sobre el estado de la partida	Estructura	2.1
D-04	Servidor Godot dedicado en VPS y dominio compartido	Estructura	ADR-04
D-05	Separación entre dominio, transporte, persistencia e interfaz	Estructura	1.2 · 5.1
D-06	Comunicación híbrida: HTTP/HTTPS y WebSocket seguro	Comunicación	3.4.3
D-07	Versionado del estado e idempotencia de acciones	Consistencia	2.4
D-08	Autenticación por sesión y autorización por operación	Seguridad	3.4.4
D-09	Catálogo común de errores	Comunicación	3.4.3

ID	Decisión	Tema	Sección
D-10	Visibilidad restringida de la información	Seguridad	3.4.4
D-11	Desconexión y reconexión	Resiliencia	2.2.3
D-12	Turnos automáticos y eliminación por inactividad	Resiliencia	2.2.3
D-13	Persistencia del estado de partida	Persistencia	2.2.3
D-14	Historial de acciones	Persistencia	2.2.3
D-15	Recuperación y límites de resiliencia	Resiliencia	4.4
D-16	Ciclo de vida de salas y partidas	Persistencia	2.2.3
D-17	Finalización autoritativa e idempotente	Consistencia	2.4
D-18	Gestión autoritativa del mazo	Seguridad	—
D-19	Configuración y despliegue independientes	Operación	—
D-20	Logging técnico	Operación	—
D-21	Compatibilidad del protocolo y actualización del cliente	Comunicación	—
D-22	Estrategia mínima de pruebas	Calidad	—
D-23	Gobernanza y comprobaciones arquitectónicas automáticas	Calidad	—

4. Análisis de Riesgo Arquitectónico y Risk Storming

4.1 Matriz de Riesgo 3x3 (Impacto vs. Probabilidad)

Se aplicó la escala de 1 a 9 (Impacto: 1-Low, 2-Med, 3-High × Probabilidad: 1-Low, 2-Med, 3-High):

- **1 - 2:** Riesgo Bajo (Verde)
- **3 - 4:** Riesgo Medio (Amarillo)
- **6 - 9:** Riesgo Alto (Rojo)

Área / Componente Evaluado	Impacto	Probabilidad	Total	Clasificación y Criterio
HandEvaluator (Algoritmo de combinaciones)	3 (High)	2 (Med)	**6**	**Alto** (Riesgo de falla combinatoria en validaciones)
GameState (Persistencia y acumulación a 101 pts)	3 (High)	2 (Med)	**6**	**Alto** (Inconsistencia en acumulación de puntajes)
Un plug-in lento bloquea el hilo principal (ADR-03 + ADR-05)	3 (High)	2 (Med)	**6**	**Alto** (Retraso en el rendimiento y en la respuesta en tiempo real)

TurnOrchestrator (Gestión de turnos y retardos)	2 (Med)	2 (Med)	**4**	**Medio** (Abandono o bloqueo temporal de turno)
El contrato v1 no alcanza para una variante futura (*volatile core*)	2 (Med)	2 (Med)	**4**	**Medio** (Habría que cambiar el núcleo para variantes con otra estructura de turno)
Plug-in malicioso o corrupto (sin *sandbox*)	3 (High)	1 (Low)	**3**	**Medio** (Vulnerabilidades e integridad del software expuestas)
UI_Rendering (Sincronización visual de cartas)	1 (Low)	1 (Low)	**1**	**Bajo** (Estilo visual y presentación)

4.2 Reporte de la Sesión de Risk Storming

El equipo ejecutó el proceso colaborativo de 3 fases:

1. **Fase 1 (Identificación Individual):** Los integrantes examinaron de manera independiente el diagrama de componentes, detectando vulnerabilidades en el procesamiento concurrente de manos y posibles pérdidas de estado ante refrescos de navegador.
2. **Fase 2 (Consenso):** Se unificaron criterios y se reclasificaron los riesgos, situando en zona roja (puntuación de 6) al motor de evaluación de manos, al sistema de persistencia acumulativa y a la ejecución de plug-ins en el hilo principal.
3. **Fase 3 (Mitigación):** Se establecieron contramedidas estructurales directas sobre el código y los límites de componentes.

4.3 Plan de Mitigación de Riesgos Críticos (Zonas 6 a 9)

- **Mitigación para HandEvaluator (Riesgo 6):** Implementación obligatoria de una suite exhaustiva de pruebas unitarias con matrices de casos combinatorios extremos (escaleras, ternas mixtas, comodines y cuartas) con cobertura en base al DA-02.
- **Mitigación para GameState / Persistencia (Riesgo 6):** Uso de transacciones atómicas de base de datos con control de concurrencia optimista y validación de estados de ronda previos a la liquidación de puntos.
- **Mitigación para plug-in lento (Riesgo 6):** cada plug-in debe pasar una prueba de contrato con p95 ≤ 150 ms en validate_closing() (RNF 2) y respetar el límite de CC ≤ 10 (FF-05) antes de instalarse; un plug-in que no cumpla no se registra.

4.4 Riesgos Aceptados y Exclusiones del MVP

Para mantener el alcance proporcionado a una entrega demostrativa, las decisiones de la sección 3.4 excluyen explícitamente los siguientes elementos. Se documentan como

limitaciones conocidas que se asumen de forma consciente y que no se mitigan con infraestructura adicional en el MVP.

Exclusión del MVP	Decisión	Alcance de la limitación
Microservicios	D-01, D-06	El sistema se despliega como una sola aplicación de servidor (monolito modular).
Firmas criptográficas personalizadas por solicitud	D-08	La seguridad se apoya en HTTPS/WSS, autenticación de sesión, validación de reglas y controles de autorización.
Redundancia, replicación o alta disponibilidad de la base de datos; recuperación garantizada ante pérdida o corrupción	D-15	No se garantiza la recuperación ante pérdida o corrupción de la base de datos. La recuperación tras un reinicio del servidor depende de que los datos persistidos sigan disponibles.
Plataforma externa de monitorización o agregación de logs	D-20	El análisis se apoya en los logs técnicos del servidor y en el historial de acciones de la base de datos.
Negociación compleja de versiones o soporte simultáneo de varias generaciones del cliente	D-21	Los clientes incompatibles se rechazan con un error explícito y se les solicita actualizar o recargar.
Event sourcing completo	D-14	El historial es complementario al estado actual; no se reconstruye obligatoriamente una partida reproduciendo todos los eventos.

5. Gobernanza y Funciones de Idoneidad (Architectural Fitness Functions)

5.1 Reglas de Gobernanza Estructural y Modularidad

Para evitar la degradación estructural y el acoplamiento no deseado durante el desarrollo del MVP, se configuran reglas automatizadas de arquitectura, conocidas como *fitness functions* (Richards & Ford, Cap. 6). Cada regla se implementa como una prueba de **GUT** (*Godot Unit Test*), el framework de pruebas de GDScript, que se ejecuta en el equipo de cada desarrollador y en integración continua, y falla con un mensaje que identifica el archivo y la línea responsables. La tabla resume las cinco reglas implementadas en el repositorio; su código completo se adjunta en el Anexo A.

ID	Regla	Qué comprueba	Origen
FF-01	Aislamiento de capas	FF-01a: la presentación no accede a la persistencia. FF-01b: la presentación no entra al núcleo directamente; solo a través de la sesión de red, que reenvía cada comando al orquestador. El núcleo no depende de la presentación, la red ni la persistencia concreta, ni referencia de forma estática los plug-ins de `res://rules/modes/`.	Regla 5.1 y D-05
FF-02	Independencia de modalidades	Ningún archivo de la modalidad clásica depende de uno de Apuntado, ni a la inversa: no hay dependencias cruzadas ni cíclicas entre las reglas específicas.	Regla 5.1

ID	Regla	Qué comprueba	Origen
FF-03	Ausencia de ciclos	El grafo de dependencias entre módulos es acíclico.	Regla 5.1, ampliada a todos los módulos
FF-04	Núcleo libre de infraestructura	El núcleo no usa clases de interfaz gráfica, de transporte (HTTP, WebSocket, RPC) ni de acceso directo a la persistencia.	D-05
FF-05	Complejidad ciclomática	Ninguna función del motor de reglas supera una complejidad ciclomática (CC) de 10.	D-23

Los módulos evaluados son los del diagrama de componentes (sección 1.3) y el módulo shared del catálogo común de errores (D-09). Se complementan con una prueba de sanidad que verifica que el análisis realmente encuentra código y con pruebas del propio analizador, para evitar que una regla pase sin comprobar nada.

5.2 Implementación y Código de Fitness Functions

5.2.1 Por qué GUT

- **Mismo lenguaje y motor que el juego:** las reglas se escriben en GDScript y se ejecutan con Godot, sin intérpretes ni herramientas externas en integración continua.
- **Ejecución sin interfaz y código de salida:** con --headless, el proceso termina con código distinto de cero si una regla falla, lo que permite bloquear la fusión de código que viole la arquitectura.
- **Resultados en JUnit XML:** el formato estándar que consumen las plataformas de integración continua.
- **Versión fijada:** GUT está atado a la versión del motor. Se usa GUT 9.5.0 con Godot 4.5.x (9.6.1 para 4.6.x y 9.7.1 para 4.7.x) y se versiona en addons/gut para que todo el equipo y la integración continua usen la misma.

5.2.2 Estructura en el repositorio

```
tests/
├── architecture/                 fitness functions (D-23)
│   ├── support/
│   │   ├── arch_config.gd        configuración (único archivo a editar)
│   │   └── arch_scanner.gd       análisis estático del código
│   ├── test_fitness_00_sanity.gd  el análisis ve código
│   ├── test_fitness_01_capas.gd   FF-01 y FF-04
│   ├── test_fitness_02_dependencias.gd   FF-02 y FF-03
│   ├── test_fitness_03_complejidad.gd    FF-05
│   └── test_arch_support.gd      pruebas del propio analizador
├── unit/                         pruebas de D-22
│   ├── rules/   server/   fallback/
└── results/                      resultados JUnit XML
```

5.2.3 Cómo funciona el análisis

El analizador recorre los scripts y escenas del proyecto y detecta las dependencias de cada archivo: rutas `res://` y `uid://` literales (`preload`, `load`, `extends`), nombres de `class_name`, autoloads de `project.godot` y referencias de las escenas `.tscn`. Ignora comentarios y el contenido de los textos. Luego agrupa los archivos por módulo según la configuración y construye un grafo sobre el que consultan las fitness functions.

Archivo	Responsabilidad
`arch_config.gd`	Único archivo de configuración: carpetas de cada módulo, dependencias prohibidas, grupos de modalidades, identificadores prohibidos en el núcleo, límite y alcance de la complejidad.
`arch_scanner.gd`	Análisis estático: limpieza del código, construcción del grafo, búsqueda de ciclos y cálculo de la complejidad ciclomática.
`test_fitness_01` a `03`	Las cinco fitness functions (FF-01 a FF-05), cada una como una prueba GUT.
`test_fitness_00_sanity.gd`	Falla si el análisis no encuentra código y avisa de los módulos declarados que no tienen archivos.
`test_arch_support.gd`	Pruebas del analizador con texto en memoria: dependencias, ciclos, modalidades y complejidad.

5.2.4 Ejemplo: FF-01, aislamiento de capas

Cada fitness function es una prueba corta que delega el análisis en el escáner. FF-01 comprueba las dependencias prohibidas definidas en la configuración:

```
extends GutTest

const Config := preload("res://tests/architecture/support/arch_config.gd")
const Scanner := preload("res://tests/architecture/support/arch_scanner.gd")

var _graph: Dictionary

func before_all() -> void:
    _graph = Scanner.cached_graph(Config.MODULES, Config.EXCLUDED_PATHS)

func test_ff01_la_presentacion_y_el_nucleo_respetan_las_capas() -> void:
    var violations := Scanner.forbidden_dependencies(
            _graph, Config.forbidden_dependencies())
    assert_true(violations.is_empty(),
        "FF-01 violada (%d):\n  %s" % [violations.size(), "\n  ".join(violations)])
```

Si alguien introduce, por ejemplo, una referencia de la interfaz al repositorio SQL, la prueba falla y señala el origen exacto:

```
FF-01 violada (1):
  [presentation] no debe depender de [infrastructure.persistence]:
    res://presentation/game_table_ui.gd ->
      res://infrastructure/persistence/sql_match_repository.gd
```

FF-02 a FF-05 siguen el mismo patrón y se incluyen completas en el Anexo A. El mecanismo se validó sobre un proyecto de muestra con la misma estructura modular y con Godot 4.5 y GUT 9.5.0: sin violaciones pasan todas las pruebas y, al introducir violaciones deliberadas en cada regla, todas fallan y el proceso termina con código 1.

5.2.5 Ejecución y alcance

Tras clonar el repositorio (o borrar `.godot/`) debe importarse el proyecto antes de ejecutar GUT; sin ese paso Godot no reconoce los `class_name` del proyecto:

```
godot --headless --import
godot --headless -s res://addons/gut/gut_cmdln.gd \
      -gdir=res://tests/architecture/ -ginclude_subdirs -gexit
```

En integración continua el mismo comando se ejecuta como un paso independiente de las pruebas unitarias, de modo que una violación de arquitectura se distingue de un fallo funcional y detiene la fusión. Limitaciones del análisis estático: El código de Clásica y Apuntado sí se analiza porque vive en el repo; solo quedan fuera los `.pck` de terceros

5.3 Límites de Complejidad Ciclomática y Reglas de Dependencias

5.3.1 Complejidad ciclomática (FF-05)

Para garantizar métodos atómicos y fáciles de probar, ninguna función del motor de reglas puede superar una complejidad ciclomática (CC) de **10** (D-23). El analizador calcula la CC de cada función como 1 más el número de puntos de decisión:

Construcción	Aporte a la CC
`if`, `elif` (incluye el operador ternario)	+1 por cada uno
`for`, `while`	+1 por cada uno
`and`, `or` (&&, ||)	+1 por cada operador
Ramas de `match`	+1 por rama, sin contar la comodín _
`else`, comentarios y contenido de textos	No suman

El alcance inicial es `res://rules/`, que incluye los evaluadores de combinaciones; se amplía a otros módulos a medida que su código cumpla el límite. Una función que lo supera hace fallar FF-05 e indica archivo, línea y valor, y el proceso de integración continua se detiene:

```
FF-05 violada (1 funciones con CC > 10):
  res://rules/hand_evaluation.gd:13  demasiado_complejo()  CC=14  (límite 10)
```

La medida es una aproximación propia del proyecto, que puede diferir en una unidad de la de otras herramientas; se usa como límite relativo para mantener las funciones pequeñas, no como medida absoluta.

5.3.2 Reglas de dependencias (FF-01 a FF-04)

La siguiente matriz resume las dependencias que el analizador rechaza.

Módulo o grupo	No debe depender de	Fundamento
presentation	infrastructure.persistence	Regla 5.1: la interfaz no accede a la persistencia (FF-01a).

Módulo o grupo	No debe depender de	Fundamento
presentation	Núcleo (orchestration, match, turn, scoring, deck, notification, rules)	Regla 5.1: la interfaz solo entra al núcleo por infrastructure.network, que reenvía los comandos al orquestador (FF-01b).
Núcleo: match, turn, scoring, rules, deck, orchestration, notification	presentation, infrastructure.network, infrastructure.persistence	D-05: el dominio no depende de interfaz, transporte ni persistencia concreta (FF-01).
Núcleo, incluido rules/selection	rules/modes/	ADR-03: el núcleo solo conoce GameRule; RuleRegistry carga los plug-ins leyendo plugin.cfg (FF-01).
`shared`	Cualquier otro módulo	D-09: el catálogo común no tiene dependencias (FF-01).
Archivos de `rules/modes/classic*` y de `rules/modes/apuntado*`	Entre sí	Regla 5.1: modalidades independientes (FF-02).
Todos los módulos	Ciclos entre módulos	Grafo acíclico (FF-03).

FF-04 prohíbe además en el núcleo los identificadores de interfaz (`Control`, `CanvasItem`, `Node2D`, `Node3D`), de transporte (`HTTPRequest`, `HTTPClient`, `WebSocketPeer`, `WebSocketMultiplayerPeer`, `ENetMultiplayerPeer`, `MultiplayerPeer`, `MultiplayerAPI`) y la anotación `@rpc`.

Todas las reglas se ajustan en el único archivo `arch_config.gd`. Relajar una regla es una decisión de arquitectura: debe justificarse y revisarse en el *pull request*.

5.4 Estrategia Mínima de Pruebas y Comprobaciones Arquitectónicas

Esta subsección desarrolla las decisiones **D-22** (estrategia mínima de pruebas) y **D-23** (gobernanza y comprobaciones automáticas) de la sección 3.4.

D-22 · Estrategia mínima de pruebas

Decisión: El MVP debe contar con tres grupos mínimos de pruebas, que deben existir y pasar antes de considerar funcional la entrega. No se establece un porcentaje arbitrario de cobertura.

Grupo	Alcance mínimo
1. Pruebas unitarias del motor de reglas	Validación de las reglas de ambas variantes, puntuación, eliminaciones, robos, descartes y condiciones de finalización. Incluye la regla de inicio de la ronda siguiente cuando se elimina al jugador que cerró (3.4.7).
2. Pruebas de validación cliente-servidor	Acciones fuera de turno, acciones ilegales, participantes eliminados, usuarios no autorizados, versiones obsoletas y reintentos duplicados.
3. Pruebas de *fallback*	Desconexión, vencimiento del temporizador, eliminación por inactividad, desincronización, reconexión y prevención de la resolución duplicada de un turno.

D-23 · Gobernanza arquitectónica y comprobaciones automáticas

Decisión: Se mantienen comprobaciones para proteger los límites entre módulos, las dependencias permitidas y la complejidad del código. El proyecto contempla:

- Pruebas de arquitectura con GDScript/GUT.
- El directorio `/tests/architecture/` para alojarlas.
- Un límite de complejidad ciclomática de 10.